

A.Tejaswini-192471043

101.Explain why Hamiltonian Path and Hamiltonian Cycle problems are computationally equivalent in complexity.

Ans:

Q101. Hamiltonian Path and Hamiltonian Cycle

```
def is_safe_path(vertex, graph, path):
```

```
    if vertex in path:
```

```
        return False
```

```
    if not path:
```

```
        return True
```

```
    last_vertex = path[-1]
```

```
    if graph[last_vertex][vertex] == 0:
```

```
        return False
```

```
    return True
```

```
def hamiltonian_path_util(graph, path, n):
```

```
    if len(path) == n:
```

```
        return True
```

```
    for vertex in range(n):
```

```
        if is_safe_path(vertex, graph, path):
```

```
            path.append(vertex)
```

```
            if hamiltonian_path_util(graph, path, n):
```

```
                return True
```

```
            path.pop()
```

```
    return False
```

```
def hamiltonian_path(graph):
```

```
    n = len(graph)
```

```

for start in range(n):
    path = [start]
    if hamiltonian_path_util(graph, path, n):
        return path
    return None

def is_safe_cycle(vertex, graph, path):
    if vertex in path:
        return False
    last_vertex = path[-1]
    if graph[last_vertex][vertex] == 0:
        return False
    return True

def hamiltonian_cycle_util(graph, path, n):
    if len(path) == n:
        last_vertex = path[-1]
        first_vertex = path[0]
        if graph[last_vertex][first_vertex] == 1:
            return True
        return False
    for vertex in range(1, n):
        if is_safe_cycle(vertex, graph, path):
            path.append(vertex)
            if hamiltonian_cycle_util(graph, path, n):
                return True
            path.pop()
    return False

def hamiltonian_cycle(graph):

```

```
n = len(graph)
path = [0]
if hamiltonian_cycle_util(graph, path, n):
    path.append(path[0])
    return path
return None

# Example graph
graph = [
    [0, 1, 1, 0],
    [1, 0, 1, 1],
    [1, 1, 0, 1],
    [0, 1, 1, 0]
]

path_result = hamiltonian_path(graph)
cycle_result = hamiltonian_cycle(graph)
print("Hamiltonian Path:")
if path_result:
    print(path_result)
else:
    print("No Hamiltonian Path exists")
print("\nHamiltonian Cycle:")
if cycle_result:
    print(cycle_result)
else:
    print("No Hamiltonian Cycle exists")
```

102. Analyze the time complexity of solving Sudoku using backtracking in the worst case.

Ans:

Q102. Sudoku Solver using Backtracking

```
def print_board(board):  
    for row in board:  
        print(row)  
  
def is_valid(board, row, col, number):  
    # Check row  
    for j in range(9):  
        if board[row][j] == number:  
            return False  
    # Check column  
    for i in range(9):  
        if board[i][col] == number:  
            return False  
    # Check 3x3 box  
    start_row = row - row % 3  
    start_col = col - col % 3  
    for i in range(start_row, start_row + 3):  
        for j in range(start_col, start_col + 3):  
            if board[i][j] == number:  
                return False  
    return True  
  
def find_empty_cell(board):  
    for i in range(9):  
        for j in range(9):  
            if board[i][j] == 0:
```

```

return i, j
return None

def solve_sudoku(board):
    empty_cell = find_empty_cell(board)
    if empty_cell is None:
        return True
    row, col = empty_cell
    for number in range(1, 10):
        if is_valid(board, row, col, number):
            board[row][col] = number
            if solve_sudoku(board):
                return True
            board[row][col] = 0
    return False

# Sample Sudoku puzzle
board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],

    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],

    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]

```

```
]
```

```
print("Original Sudoku:")
```

```
print_board(board)
```

```
if solve_sudoku(board):
```

```
    print("\nSolved Sudoku:")
```

```
    print_board(board)
```

```
else:
```

```
    print("No solution exists")
```